Node Talker Overview

What Node Talker is, what it connects to on the node, how audio comes in and goes out, and where each part of the rest of this series begins.

RX PATH | TX PATH | BALANCER | AUDIO SETTINGS

SUMMARY · READ-ONLY REVIEW

A phone or laptop page that talks to the node

Node Talker serves a web page. The page listens to the node's receive audio and sends the operator's voice back onto the node's transmit path. It is a common tool: the same file serves both builds, and it is installed by its own `--install`.

ITEM	VALUE	SET IN
Web port	8993 (HTTPS by default)	`port`, `https`
Audio in from the node	UDP 127.0.0.1:6200	`udp_port`
Transmit node	Node 1921, a USRP channel on ports 32018 (send) and 34018 (listen)	Built into the tool; written to rpt.conf
Listener slots	3 by default (1 to 10)	`max_listeners`
Config file	`/etc/asl_dvs/nodetalker.conf`, section `[nodetalker]`	Written by the page's saves

The config file keeps the `asl_dvs` name, as the naming rules require. The older name, nodelisten, is still read as a fallback section.

Audio goes both ways

2.1 Receive (node to phone)

1. Asterisk sends the node's receive audio to a feed command (`outstreamcmd`) written into rpt.conf for the chosen node.
2. The feed command reads that audio and sends it as UDP datagrams to 127.0.0.1:6200.
3. Node Talker turns the datagrams into 20 ms frames, cleans them, and queues them for each listener.
4. Each listener reads a stream over HTTP, as u-law by default, or as 16-bit PCM when asked.

Part 2 of this series covers each step.

2.2 Transmit (phone to node)

1. The page opens a WebSocket and sends the operator's microphone audio as 16-bit PCM.
2. When the operator keys up, the server checks the node is ready and linked, and that the channel is not busy.
3. Audio is paced out at 20 ms a frame as USRP packets to Asterisk on the transmit node.
4. Asterisk keys the node's transmitter while the packets flow, and drops the key when they stop.

Part 3 of this series covers each step.

2.3 The balancer and the audio settings

The balancer is a set of tone and live checks that the page runs, and it changes settings one at a time. The settings control gain, filtering, the listen buffer and the transmit buffer. Part 4 covers both.

The main parts of the code

PART	WHAT IT IS	WHERE TO START
Receive hub	Takes UDP audio, makes frames, sends them to listeners	`Hub`, `udp_loop`, `run_feed`
Receive cleanup	Filter, gain, AGC and gate on the audio	`RxClean`
Receive health	Records gaps, jitter and listener stalls, with a verdict	`RxHealth`, `rxh_verdict`
Transmit	Keys the node, buffers and paces the audio	`Talker`
Transmit setup	Writes the node 1921 block into rpt.conf	`tx_enable`, `tx_state`
Load guard	Stops tone tests when the Pi is stressed	`LoadGuard`
Balancer	Saves, restores and keeps a balance run	`balance_snapshot`, `balance_restore`, `balance_keep`
Page	The browser code that runs the tests and the balancer	The script in the page, from `expRun`

What runs inside the service

4.1 HTTPS

With `https` on (the default), the service creates a self-signed certificate on first start with `openssl` (elliptic-curve key, valid 10 years, names for the host, `host.local` and the node's LAN addresses) and serves TLS 1.2 or later on the one port. A plain HTTP request to that port is answered with a 301 redirect to the same address over HTTPS. If the certificate cannot be made or loaded, the service falls back to plain HTTP and the page shows a warning that the password travels in plain text. Push-to-talk needs HTTPS in the browser, so the log says so.

4.2 The threads

After the listener is up, the service starts these background threads: the Asterisk key-up watcher, the audio pump (20 ms), the UDP receiver, the transmit pacer (20 ms), the transmit drain, the link keeper, the link guard and link log, the IAX and Wi-Fi health samplers, the Pi health sampler, and the systemd watchdog. It then tells systemd it is ready (`Type=notify`).

4.3 Stopping

On a stop signal, the service ends any transmission in progress and, if the transmit node is set up, unlinks node 1921 from the main node, so a stopped service does not leave the transmit node linked.

Access, briefly

- **Local network by default.** A request from outside the local network is treated as coming from the internet and is refused on the gated routes, unless `allow_wan` is on. The default is off.
- **Login.** The page uses a login with session cookies and lockouts. The security series covers these. Only the page itself, `/api/health` and `/api/whoami` answer without a login.
- **Public listening.** With `public_listen` on (off by default), the status and audio stream routes (`/api/status`, `/api/listen`) also answer without a login, to anyone who passes the local-network test. Talking (`/api/talk`) always needs a login.
- **Proxied requests** from a loopback address that carry a proxy header are treated as outside.

What the overview shows

6.1 Two streams, one node

Receive and transmit use different ports and different pieces of Asterisk. A problem on one side does not necessarily affect the other, and the logs and health readings keep them apart.

6.2 Node 1921 is a separate node

The transmit path is a second node in rpt.conf, linked to the main node. The receive path uses the main node's own output hook. Operators must restart Asterisk after the rpt.conf change that the setup makes.

6.3 Not checked

- The full set of routes in the page (only the main structure was read).
- The Asterisk side of the USRP channel on a running node.
- The page's phone-app path in full.